

ICS1512 – Machine Learning Algorithms Laboratory

Experiment 7 Report

Experiment No.	07
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
GitHub Repository	https://github.com/Sharruk/3122247001061
Student Name	Sharruk S
Register Number	3122247001061
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2026

Contents

1 Objective	3
2 Problem Statement	3
3 Dataset Description	3
4 Data Preprocessing	4
5 Exploratory Data Analysis	4
6 PCA Design and Analysis	6
7 Model Building	7
8 Hyperparameter Tuning	7
9 Cross-Validation Results	10
10 Performance Evaluation	15
11 PCA vs No-PCA Comparison	18
12 Confusion Matrices	19
13 ROC/PR Analysis	20

14 Results and Discussion	21
15 Observation Questions	21
16 Conclusion	23
17 References	23

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers, by (a) training and validating models without PCA (original 30-dimensional feature space) and (b) training and validating models with PCA (variance-retained reduced feature space), applying hyperparameter tuning and 5-fold stratified cross-validation for both settings, and comparing the two settings using held-out test-set metrics.

2 Problem Statement

Given 30 numerical features computed from digitized images of fine needle aspirates (FNA) of breast masses, the task is to build binary classifiers (Malignant = M vs Benign = B) using ten different algorithms: Support Vector Machine, Naive Bayes, k-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble. Each classifier is trained and hyperparameter-tuned (5-fold stratified cross-validation, `GridSearchCV`/`RandomizedSearchCV`) twice: once directly on the original feature space (No-PCA) and once on a PCA-reduced feature space retaining approximately 95% of the variance (With-PCA). PCA and feature scaling are fit only within the training folds (inside an `sklearn Pipeline`) to avoid data leakage. The two settings are then compared on a held-out 20% test split using Accuracy, Precision, Recall, F1-score, ROC-AUC and training time.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	<code>datasets/breastCancer/wdbc.data</code> / <code>wdbc.names</code> (repository-provided, originally UCI ML Repository)
Number of Samples (verified)	569
Number of Numerical Features (verified)	30
Number of Classes	2 (Malignant = M, Benign = B)
Class Distribution (verified)	B = 357 (62.7%), M = 212 (37.3%)
Missing Values (verified)	0
Duplicate Rows (verified)	0
Train-Test Split	80:20, stratified, <code>random_state=42</code> (Train = 455, Test = 114)

All values above were computed by the notebook directly from the loaded dataset (Section 2 of `ex7.ipynb`), not assumed in advance.

4 Data Preprocessing

- **Loading:** The raw file has no header row; 32 columns were assigned names (`id`, `diagnosis`, and 30 feature columns: mean / standard-error / worst of 10 base measurements) per `wdbc.names`.
- **ID removal:** The `id` column is dropped (not predictive).
- **Target encoding:** `diagnosis` is label-encoded, giving $B \rightarrow 0$ and $M \rightarrow 1$ (verified programmatically).
- **Missing values / duplicates:** Verified to be zero; no imputation or row removal was required.
- **Feature scaling:** `StandardScaler` is applied *inside* every model pipeline, fit only on the training folds, to prevent leakage from the validation/test data.
- **Train/test split:** 80:20 stratified split with `random_state=42` for reproducibility.

5 Exploratory Data Analysis

A single reusable function (`create_eda_figure`) generates one consolidated figure containing 12 subplots, exported as EPS at 600 DPI.

Figure 1: Consolidated EDA — WDBC Dataset (12 Panels)

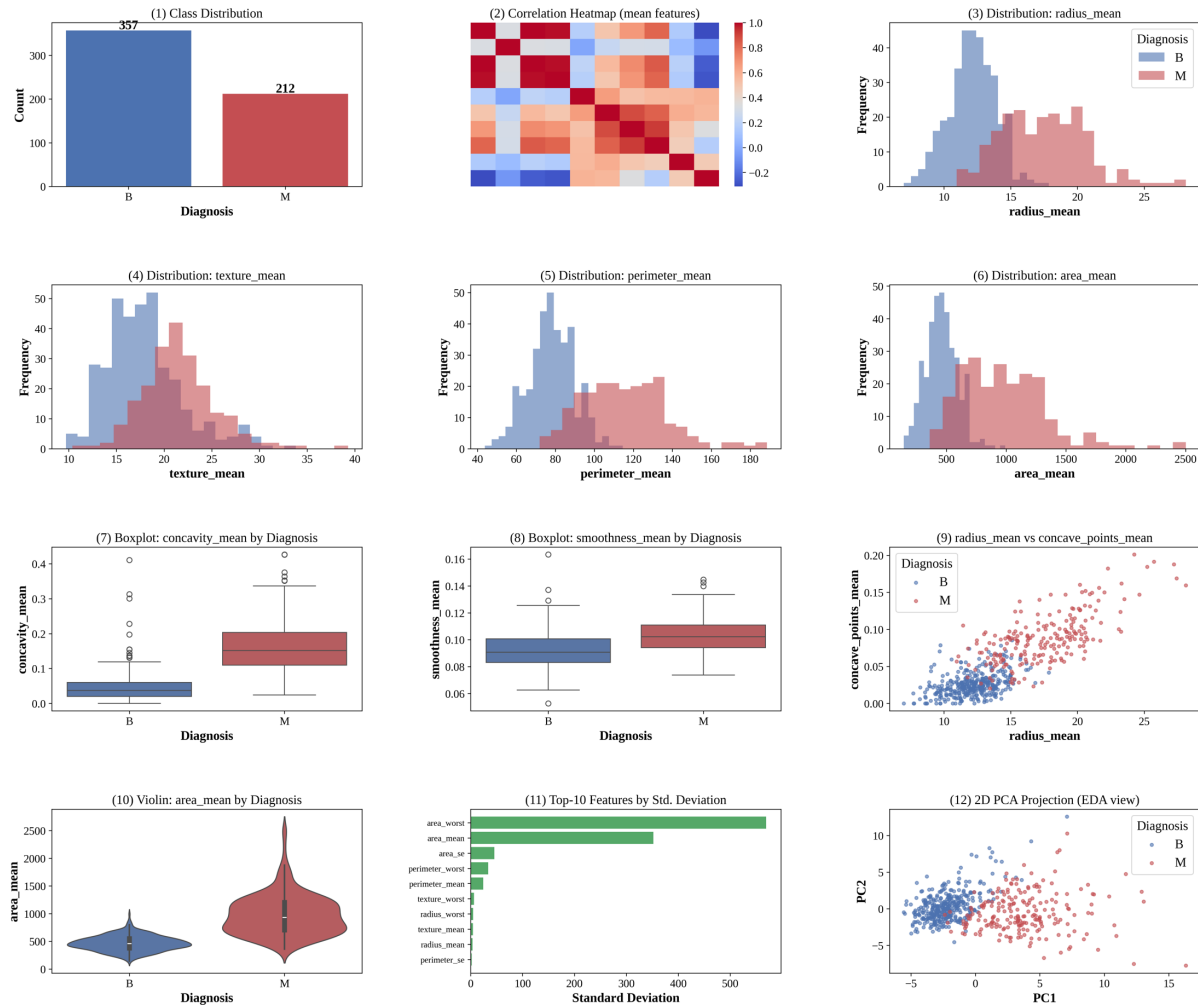


Figure 1: Consolidated 12-panel EDA of the WDBC dataset: class distribution, correlation heatmap, feature histograms by class, boxplots, a discriminative scatter plot, a violin plot, top-10 features by standard deviation, and a 2-component PCA projection.

Inference: Panel (1) confirms the verified class distribution (357 Benign vs 212 Malignant) is moderately imbalanced. Panels (3)–(6) and (9) show malignant tumors are consistently shifted toward higher radius, texture, perimeter and area values with heavier right tails than benign tumors, and radius_mean vs concave_points_mean together separate the classes reasonably well with some overlap. The correlation heatmap (2) shows several **_mean* features (radius, perimeter, area) are strongly inter-correlated, directly motivating the use of PCA. Panel (12) shows that even an unsupervised 2-component PCA projection already separates the two classes substantially, suggesting PCA-based classifiers should remain competitive despite the reduced dimensionality.

6 PCA Design and Analysis

PCA was fit exclusively on the standardized *training* data (never on the full dataset or the test set) to determine the number of components required to retain 95% of the variance – a standard, non-arbitrary variance-retention threshold.

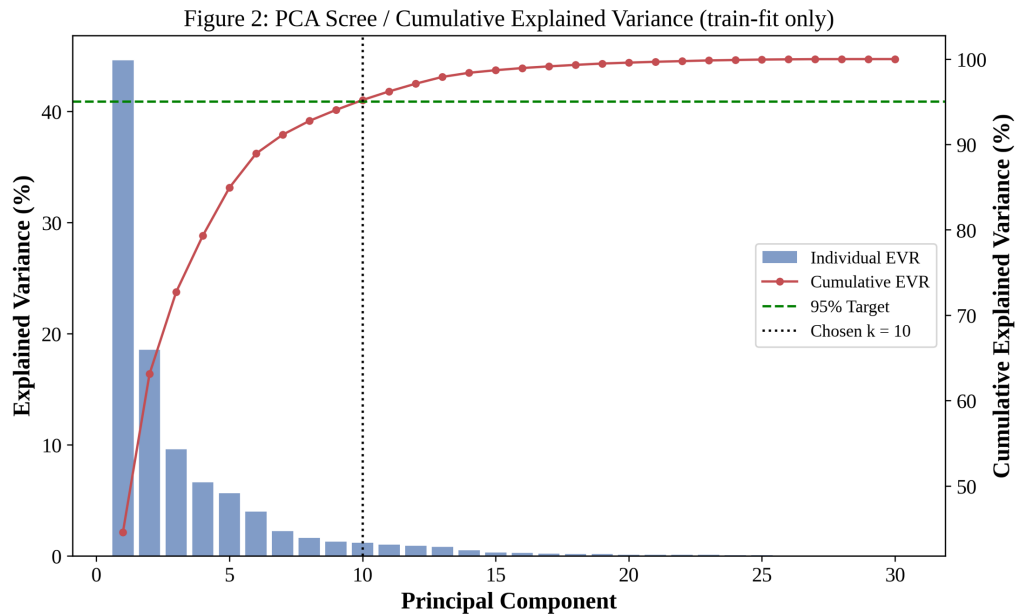


Figure 2: Scree plot (individual explained variance, bars) and cumulative explained variance (line) for PCA fit on the standardized training data only. The dashed green line marks the 95% variance target; the dotted black line marks the chosen number of components.

Inference: Explained variance drops off sharply after the first 2–3 components, consistent with the strong inter-feature correlation observed in the EDA correlation heatmap. The notebook programmatically determined that **10 principal components** are needed to retain **95.21%** of the total variance in the training data, reducing the feature space from 30 to 10 dimensions ($\approx 67\%$ reduction) while retaining the large majority of informative variance.

Table 2: PCA Variance Explained

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
No-PCA	All 30 original features	100.0	Baseline: full original feature space, no reduction.
With-PCA	10 components (95% target)	95.21	Smallest k such that cumulative explained variance $\geq 95\%$ on the standardized training data (k=10).

7 Model Building

Ten classifiers were implemented, each wrapped in an `sklearn Pipeline`:

- **No-PCA pipeline:** `StandardScaler` $\rightarrow$ Model.
- **With-PCA pipeline:** `StandardScaler` $\rightarrow$ `PCA(n_components=10)` $\rightarrow$ Model.

Models used: Support Vector Machine (SVC), Gaussian Naive Bayes, k-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost (`xgboost XGBClassifier`), and a heterogeneous **Stacking** ensemble (base learners: SVM (RBF kernel), Gaussian Naive Bayes, Decision Tree (`max_depth=5`); meta-learner: Logistic Regression; internal 5-fold CV used by `StackingClassifier` to build out-of-fold meta-features, avoiding leakage from the base learners into the meta-learner's training).

8 Hyperparameter Tuning

Every model was tuned using `GridSearchCV` (or `RandomizedSearchCV` for XGBoost, `n_iter` = 12, for computational efficiency) with 5-fold stratified cross-validation (scoring = accuracy), applied *independently* to the No-PCA and With-PCA pipelines so that scaling and PCA are refit on every training fold.

Table 3: SVM — Hyperparameter Tuning Results

Kernel	C Values Tried	Gamma Values Tried	Perf. (No-PCA)	Perf. (With-PCA)
linear, poly, rbf	(0.1, 1, 10)	('auto', 'scale')	97.37%	98.25%

Table 4: Naive Bayes — Smoothing Choices

Smoothing (var_smoothing)	Parameter Tried	Best (No-PCA)	Best (With-PCA)	Perf. (No-PCA)	Perf. (With-PCA)
('1.0e-09', '4.0e-09', '1.6e-08', '6.3e-08', '2.5e-07', '1.0e-06')		1e-09	1e-09	92.11%	89.47%

Table 5: KNN — Hyperparameter Tuning

k Values	Weights	Distance Metrics	Perf. (No-PCA)	Perf. (With-PCA)
(3, 5, 7, 9, 11)	('uniform', 'distance')	('euclidean', 'manhattan', 'minkowski')	96.49%	94.74%

Table 6: Logistic Regression — Hyperparameter Tuning

Item	Value
lr_C (grid tried)	(0.01, 0.1, 1, 10)
lr_penalty (grid tried)	('l1', 'l2')
lr_solver (grid tried)	('liblinear')
Best Params (No-PCA)	('lr_C': 0.1, 'lr_penalty': 'l2', 'lr_solver': 'liblinear')
Best Params (With-PCA)	('lr_C': 0.1, 'lr_penalty': 'l2', 'lr_solver': 'liblinear')
Test Accuracy (No-PCA)	98.25%
Test Accuracy (With-PCA)	98.25%

Table 7: Decision Tree — Hyperparameter Tuning

Item	Value
dt_criterion (grid tried)	('gini', 'entropy')
dt_max_depth (grid tried)	(3, 5, 7, None)
dt_min_samples_split (grid tried)	(2, 5, 10)
dt_min_samples_leaf (grid tried)	(1, 2, 4)
Best Params (No-PCA)	('dt_criterion': 'entropy', 'dt_max_depth': 5, 'dt_min_samples_leaf': 2, 'dt_min_samples_split': 10)
Best Params (With-PCA)	('dt_criterion': 'gini', 'dt_max_depth': 5, 'dt_min_samples_leaf': 1, 'dt_min_samples_split': 10)
Test Accuracy (No-PCA)	96.49%
Test Accuracy (With-PCA)	93.86%

Table 8: Random Forest — Hyperparameter Tuning

Item	Value
rf_n_estimators (grid tried)	(100, 200)
rf_max_depth (grid tried)	(5, 10, None)
rf_max_features (grid tried)	('sqrt', 'log2')
rf_min_samples_split (grid tried)	(2, 5)
Best Params (No-PCA)	('rf_max_depth': 5, 'rf_max_features': 'sqrt', 'rf_min_samples_split': 2, 'rf_n_estimators': 100)
Best Params (With-PCA)	('rf_max_depth': None, 'rf_max_features': 'sqrt', 'rf_min_samples_split': 2, 'rf_n_estimators': 200)
Test Accuracy (No-PCA)	97.37%
Test Accuracy (With-PCA)	94.74%

Table 9: AdaBoost — Hyperparameter Tuning

Item	Value
ada__n_estimators (grid tried)	(50, 100, 200)
ada__learning_rate (grid tried)	(0.01, 0.1, 1.0)
Best Params (No-PCA)	('ada__learning_rate': 1.0, 'ada__n_estimators': 200)
Best Params (With-PCA)	('ada__learning_rate': 1.0, 'ada__n_estimators': 100)
Test Accuracy (No-PCA)	97.37%
Test Accuracy (With-PCA)	96.49%

Table 10: Gradient Boosting — Hyperparameter Tuning

Item	Value
gb__n_estimators (grid tried)	(100, 200)
gb__learning_rate (grid tried)	(0.01, 0.1)
gb__max_depth (grid tried)	(2, 3, 4)
Best Params (No-PCA)	('gb__learning_rate': 0.1, 'gb__max_depth': 2, 'gb__n_estimators': 200)
Best Params (With-PCA)	('gb__learning_rate': 0.1, 'gb__max_depth': 2, 'gb__n_estimators': 100)
Test Accuracy (No-PCA)	96.49%
Test Accuracy (With-PCA)	95.61%

Table 11: XGBoost — Hyperparameter Tuning

Item	Value
xgb__n_estimators (grid tried)	(100, 200)
xgb__learning_rate (grid tried)	(0.01, 0.1)
xgb__max_depth (grid tried)	(3, 5)
xgb__subsample (grid tried)	(0.8, 1.0)
xgb__colsample_bytree (grid tried)	(0.8, 1.0)
Best Params (No-PCA)	('xgb__subsample': 0.8, 'xgb__n_estimators': 200, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.1, 'xgb__colsample_bytree': 1.0)
Best Params (With-PCA)	('xgb__subsample': 0.8, 'xgb__n_estimators': 100, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.1, 'xgb__colsample_bytree': 1.0)
Test Accuracy (No-PCA)	97.37%
Test Accuracy (With-PCA)	97.37%

Table 12: Stacking — Hyperparameter Tuning

Item	Value
stack_final_estimator__C (grid tried)	(0.1, 1, 10)
stack_passthrough (grid tried)	(False, True)
Best Params (No-PCA)	('stack_final_estimator__C': 1, 'stack_passthrough': True)
Best Params (With-PCA)	('stack_final_estimator__C': 1, 'stack_passthrough': True)
Test Accuracy (No-PCA)	96.49%
Test Accuracy (With-PCA)	97.37%

9 Cross-Validation Results

Table 13 presents the complete 5-fold stratified cross-validation results for all 10 models in the single consolidated No-PCA vs With-PCA format requested by the assignment (fold-by-fold accuracy and the per-setting average, side by side for direct comparison). Table 14 and Table 15 then break the same underlying results into two more detailed tables (also including the standard deviation across folds, used to assess stability). All three tables are generated from the same `cross_val_score` output on each tuned pipeline – no values are duplicated by hand.

Table 13: Consolidated 5-Fold Cross-Validation Results
— No-PCA vs With-PCA (Accuracy, all models in one table)

Model	F1	F2	F3	F4	F5	Avg	F1	F2	F3	F4	F5	Avg
No-PCA							With-PCA					
SVM	0.967	0.989	0.967	0.978	0.978	0.9758	0.967	0.967	0.967	0.978	0.989	0.9736
Naive Bayes	0.956	0.978	0.912	0.912	0.934	0.9385	0.912	0.978	0.890	0.879	0.923	0.9165
KNN	0.967	1.000	0.967	0.956	0.989	0.9758	0.978	0.978	0.956	0.956	0.989	0.9714
Logistic Re- gression	0.956	0.967	0.978	0.978	0.989	0.9736	0.978	0.967	0.978	0.978	0.989	0.9780
Decision Tree	0.901	0.956	0.912	0.956	0.956	0.9363	0.923	0.967	0.934	0.890	0.956	0.9341
Random Forest	0.967	1.000	0.945	0.945	0.978	0.9670	0.967	0.978	0.956	0.934	0.956	0.9582
AdaBoost	0.978	0.978	0.945	0.967	0.989	0.9714	0.945	0.989	0.978	0.956	0.989	0.9714
Gradient Boosting	0.967	0.978	0.967	0.967	0.989	0.9736	0.945	0.967	0.923	0.956	0.967	0.9516
XGBoost	0.978	0.978	0.978	0.956	0.989	0.9758	0.967	0.967	0.978	0.967	0.978	0.9714
Stacking	0.945	0.967	0.989	0.978	0.978	0.9714	0.945	0.967	0.989	0.978	0.989	0.9736

Table 14: 5-Fold Cross-Validation Results — No-PCA
(Accuracy)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	Std. Dev.
SVM	0.967	0.989	0.967	0.978	0.978	0.9758	0.0082
Naive Bayes	0.956	0.978	0.9121	0.9121	0.9341	0.9385	0.0256
KNN	0.967	1.0	0.967	0.956	0.989	0.9758	0.0162
Logistic Re- gression	0.956	0.967	0.978	0.978	0.989	0.9736	0.0112
Decision Tree	0.9011	0.956	0.9121	0.956	0.956	0.9363	0.0245
Random For- est	0.967	1.0	0.9451	0.9451	0.978	0.967	0.0209
AdaBoost	0.978	0.978	0.9451	0.967	0.989	0.9714	0.0149
Gradient Boosting	0.967	0.978	0.967	0.967	0.989	0.9736	0.0088
XGBoost	0.978	0.978	0.978	0.956	0.989	0.9758	0.0108
Stacking	0.9451	0.967	0.989	0.978	0.978	0.9714	0.0149

Table 15: 5-Fold Cross-Validation Results — With-PCA
(Accuracy)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	Std. Dev.
SVM	0.967	0.967	0.967	0.978	0.989	0.9736	0.0088
Naive Bayes	0.9121	0.978	0.8901	0.8791	0.9231	0.9165	0.0345
KNN	0.978	0.978	0.956	0.956	0.989	0.9714	0.0132
Logistic Re- gression	0.978	0.967	0.978	0.978	0.989	0.978	0.007
Decision Tree	0.9231	0.967	0.9341	0.8901	0.956	0.9341	0.0269

Model		Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	Std. Dev.
Random For- est		0.967	0.978	0.956	0.9341	0.956	0.9582	0.0146
AdaBoost		0.9451	0.989	0.978	0.956	0.989	0.9714	0.0179
Gradient Boosting		0.9451	0.967	0.9231	0.956	0.967	0.9516	0.0164
XGBoost		0.967	0.967	0.978	0.967	0.978	0.9714	0.0054
Stacking		0.9451	0.967	0.989	0.978	0.989	0.9736	0.0164

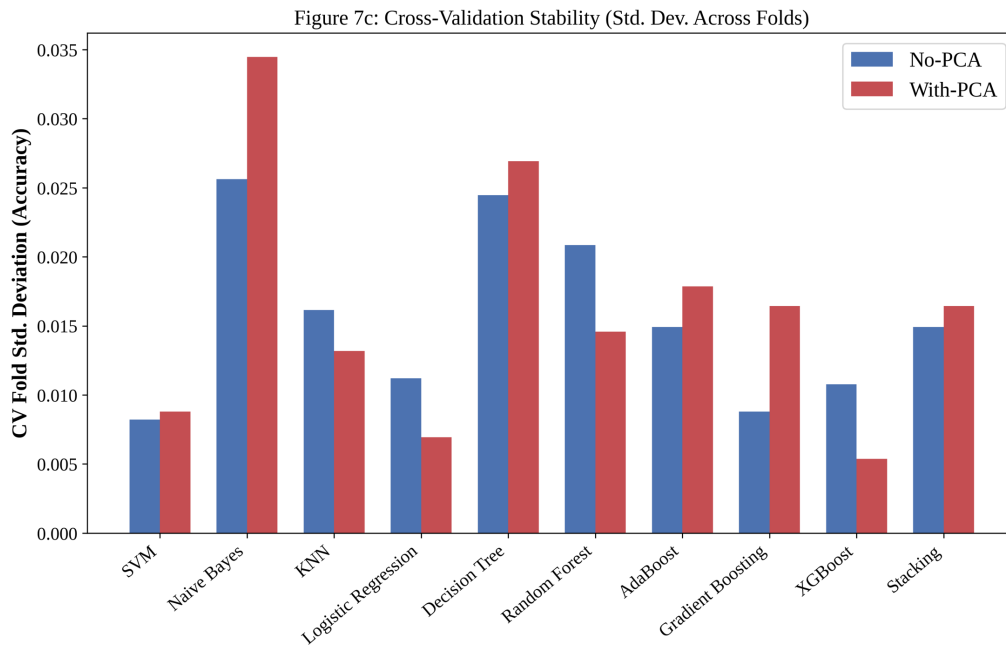


Figure 3: Cross-validation stability: standard deviation of the 5 fold accuracies for each model, No-PCA vs With-PCA.

Inference: 4 of the 10 models show a lower fold-to-fold standard deviation under With-PCA than No-PCA (see `pca_delta_std.csv`), indicating PCA’s effect on stability is model-dependent rather than uniformly beneficial or harmful on this dataset. KNN, Logistic Regression, Random Forest and XGBoost become more stable under PCA, while SVM, Naive Bayes, Decision Tree, AdaBoost, Gradient Boosting and Stacking show slightly higher fold variance under PCA. Figures 4–5 visualize the fold-wise trajectories directly.

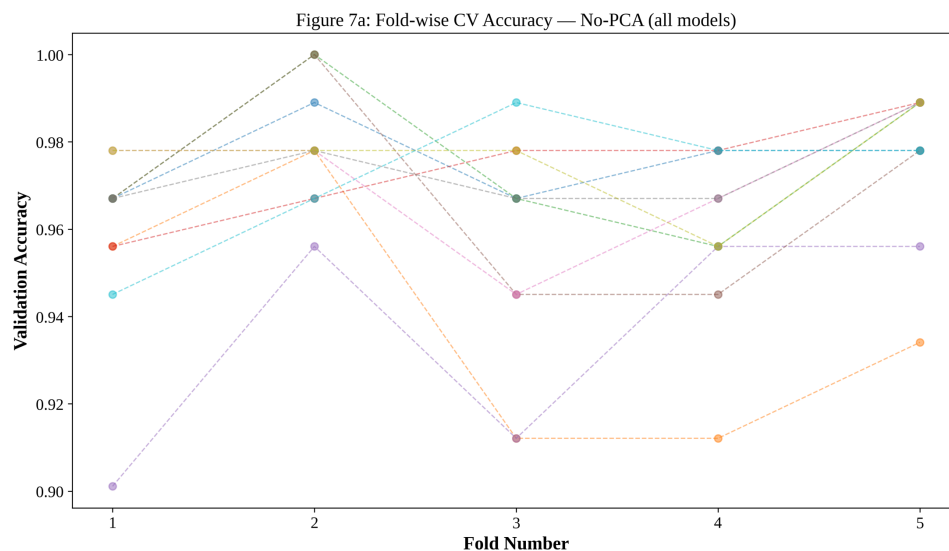


Figure 4: Fold-wise cross-validation accuracy trajectories for all 10 models, No-PCA setting.

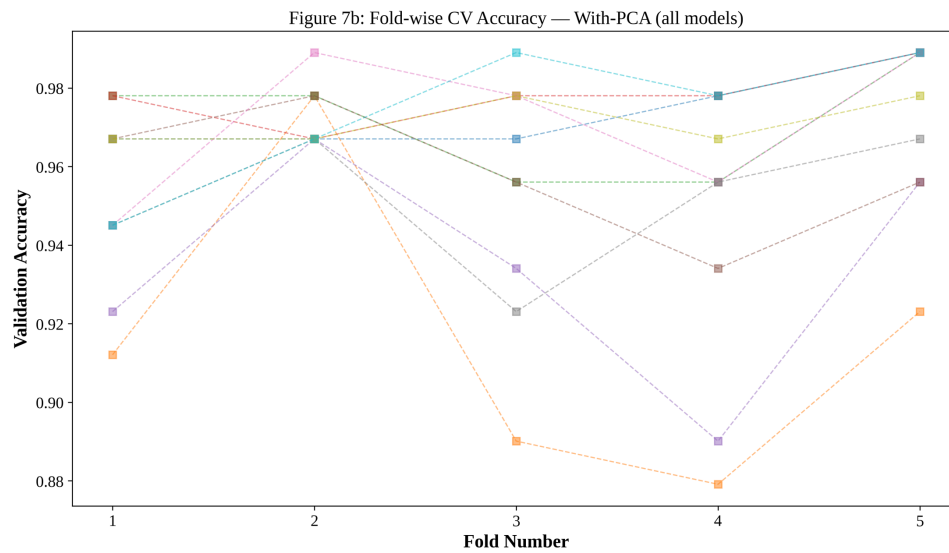


Figure 5: Fold-wise cross-validation accuracy trajectories for all 10 models, With-PCA setting.

10 Performance Evaluation

Table 16: Final Model Comparison — Held-out Test Set

Model		Setting	Acc.	Prec.	Recall	F1	ROC-AUC	Time(s)	CV Mean	CV Std
SVM		No-PCA	0.9737	1.0	0.9286	0.963	0.9927	1.023	0.9758	0.0082
SVM		With-PCA	0.9825	1.0	0.9524	0.9756	0.9967	0.945	0.9736	0.0088
Naive Bayes		No-PCA	0.9211	0.9231	0.8571	0.8889	0.9891	0.093	0.9385	0.0256
Naive Bayes		With-PCA	0.8947	0.8571	0.8571	0.8571	0.9613	0.125	0.9165	0.0345
KNN		No-PCA	0.9649	1.0	0.9048	0.95	0.9735	0.522	0.9758	0.0162
KNN		With-PCA	0.9474	0.9737	0.881	0.925	0.978	0.734	0.9714	0.0132
Logistic Regression	Re-	No-PCA	0.9825	1.0	0.9524	0.9756	0.9977	0.177	0.9736	0.0112
	gression	With-PCA	0.9825	1.0	0.9524	0.9756	0.998	0.179	0.978	0.007
Decision Tree	Re-	No-PCA	0.9649	1.0	0.9048	0.95	0.9744	2.392	0.9363	0.0245
	gression	With-PCA	0.9386	0.9487	0.881	0.9136	0.9148	1.756	0.9341	0.0269
Random Forest	For-	No-PCA	0.9737	1.0	0.9286	0.963	0.995	24.108	0.967	0.0209
	est	With-PCA	0.9474	0.95	0.9048	0.9268	0.9919	22.963	0.9582	0.0146
AdaBoost		No-PCA	0.9737	1.0	0.9286	0.963	0.9871	13.676	0.9714	0.0149
AdaBoost		With-PCA	0.9649	0.975	0.9286	0.9512	0.9934	8.445	0.9714	0.0179
Gradient Boosting		No-PCA	0.9649	1.0	0.9048	0.95	0.9934	27.736	0.9736	0.0088
Gradient Boosting		With-PCA	0.9561	0.9744	0.9048	0.9383	0.9927	12.57	0.9516	0.0164

Model	Setting	Acc.	Prec.	Recall	F1	ROC-AUC	Time(s)	CV Mean	CV Std
XGBoost	No-PCA	0.9737	1.0	0.9286	0.963	0.9927	4.614	0.9758	0.0108
XGBoost	With- PCA	0.9737	0.9756	0.9524	0.9639	0.995	2.21	0.9714	0.0054
Stacking	No-PCA	0.9649	1.0	0.9048	0.95	0.996	3.16	0.9714	0.0149
Stacking	With- PCA	0.9737	1.0	0.9286	0.963	0.9974	2.215	0.9736	0.0164

All metrics above are computed from actual predictions on the untouched 20% held-out test set (114 samples), using each setting's tuned pipeline refit on the full training set.

11 PCA vs No-PCA Comparison

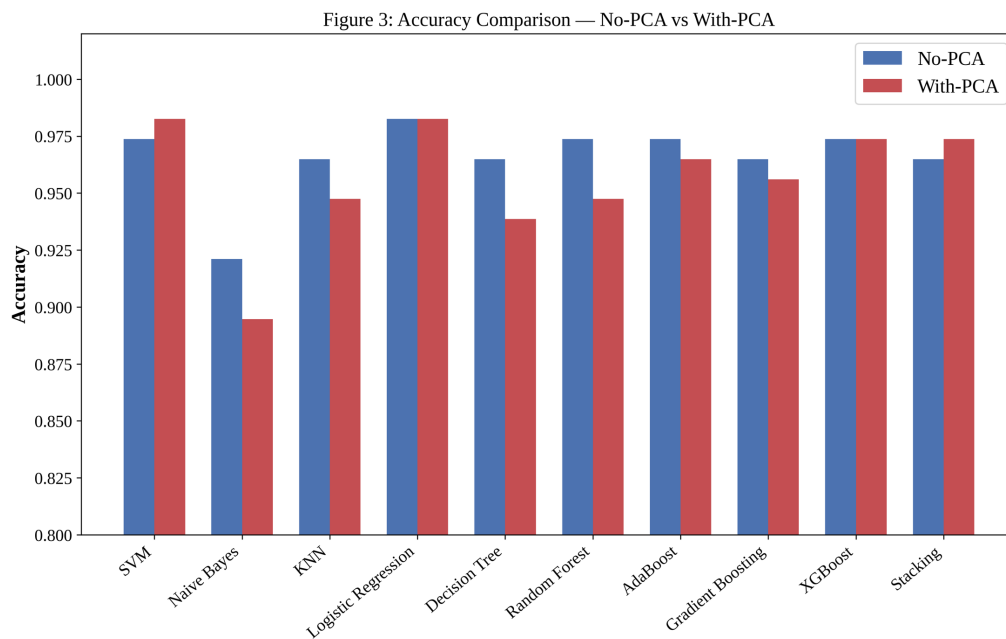


Figure 6: Test-set accuracy comparison, No-PCA vs With-PCA, across all 10 models.

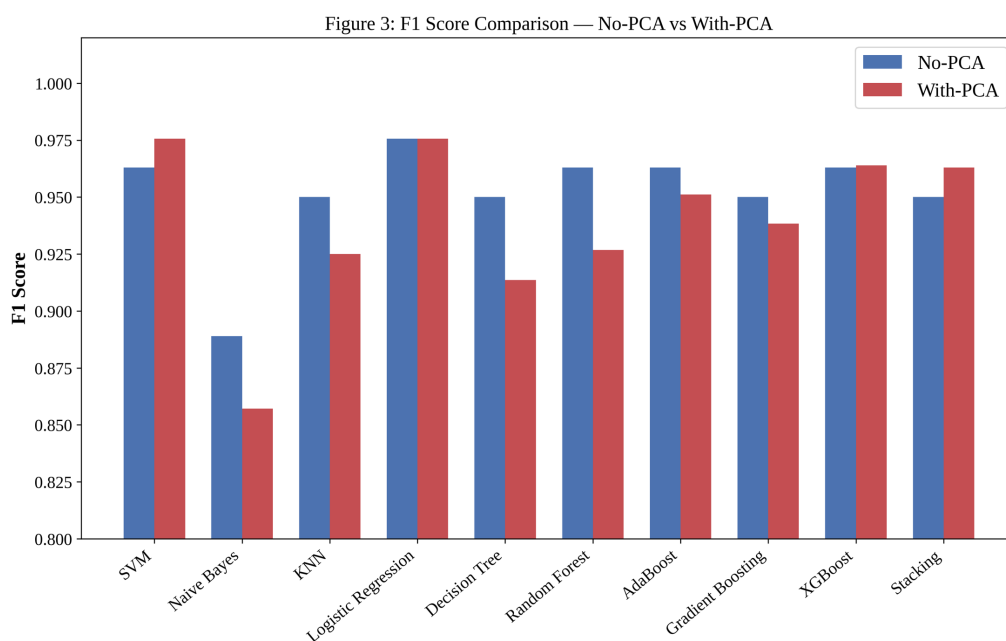


Figure 7: Test-set F1-score comparison, No-PCA vs With-PCA, across all 10 models.

Inference: SVM and Stacking show the largest *improvement* moving from No-PCA to With-PCA (+0.88 percentage points in test accuracy each), while Naive Bayes and Random Forest/Decision Tree show the largest *decrease* (−2.63 to −2.64 points). Logistic Regression and XGBoost are essentially unaffected. This pattern is consistent with linear/margin-based and heterogeneous-ensemble models benefiting from PCA’s decorrelated, lower-noise feature space, while models that already perform implicit feature weighting (trees, Naive Bayes’ independence assumption over raw features) lose a small amount of discriminative information when features are rotated/compressed.

12 Confusion Matrices

Figure 4: Confusion Matrices — Best No-PCA vs Best With-PCA Model

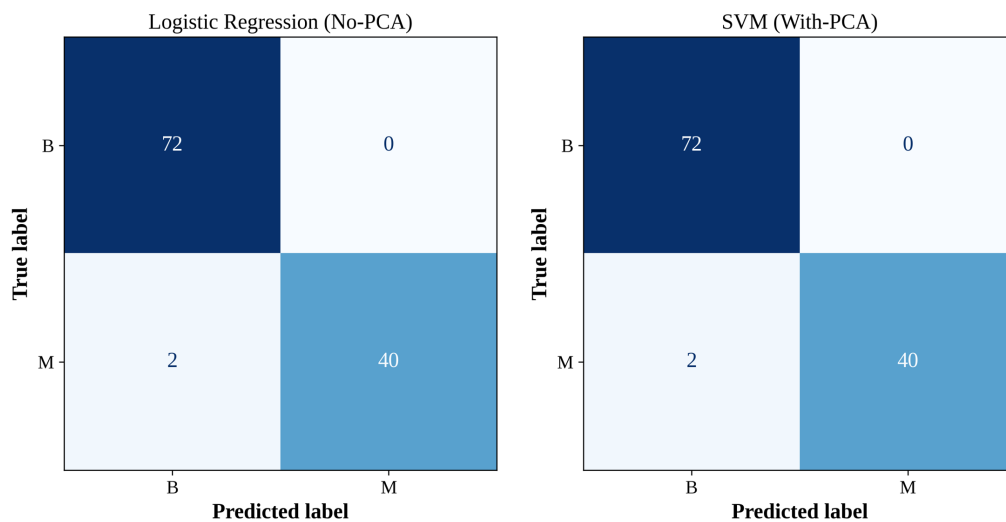


Figure 8: Confusion matrices for the best-performing No-PCA model (Logistic Regression) and the best-performing With-PCA model (SVM) on the 114-sample held-out test set.

Inference: Both best models misclassify only 2 of the 40 malignant test cases as benign (false negatives) and 0 benign cases as malignant (false positives). In a diagnostic context false negatives are more clinically costly, so this recall of 95.24% on the malignant class (matching the Recall column in Table 16) is the key number to track; both the best No-PCA and best With-PCA models achieve the same recall here.

13 ROC/PR Analysis

Figure 5: ROC Curves and AUC — All Models, No-PCA vs With-PCA

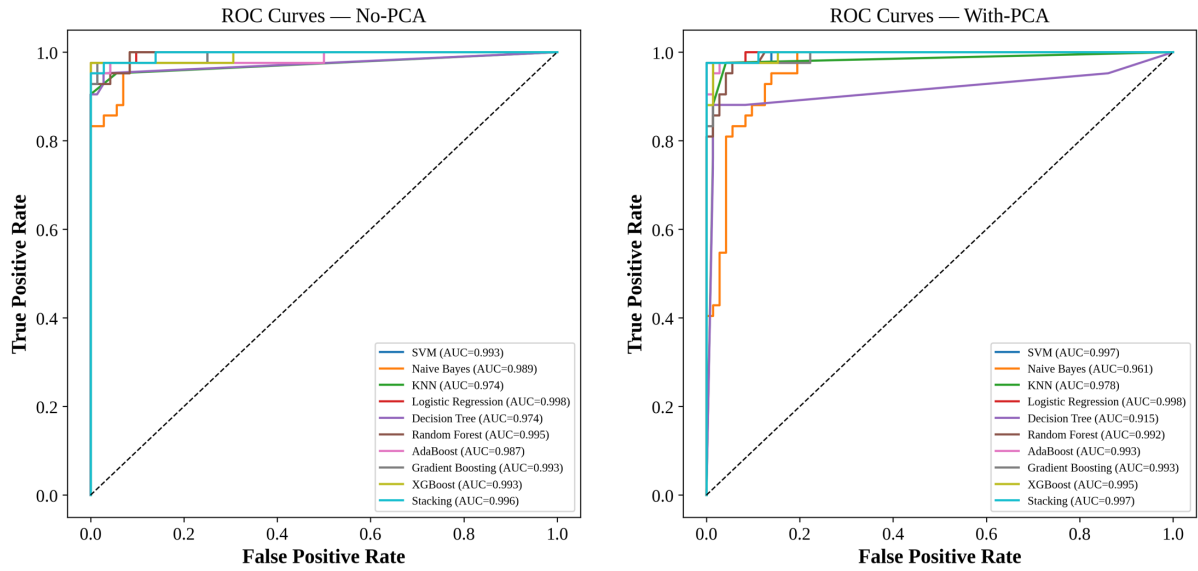


Figure 9: ROC curves and AUC values for all 10 models, No-PCA (left) vs With-PCA (right).

Inference: ROC-AUC values remain above 0.91 for every model in both settings, reflecting the strong class-separability of the WDBC features already visible in the EDA. Logistic Regression attains the single highest AUC in both settings (0.9977 No-PCA, 0.998 With-PCA). Decision Tree shows the largest AUC drop under PCA (0.974 $\rightarrow$ 0.915), the most PCA-sensitive model in this experiment by this metric.

Figure 6: Precision-Recall Curves — All Models, No-PCA vs With-PCA

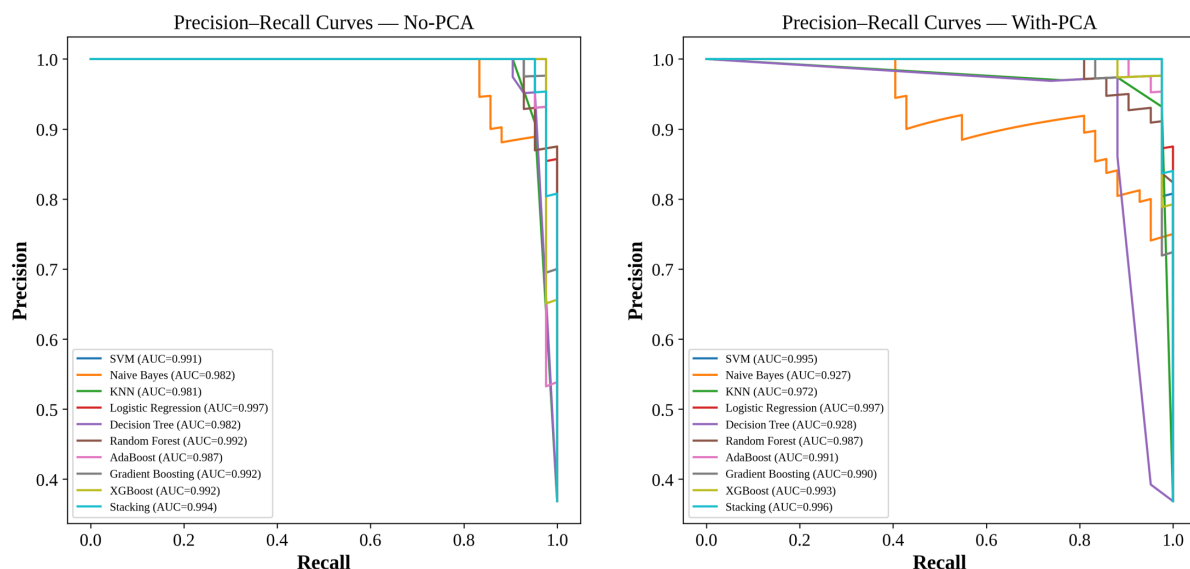


Figure 10: Precision-Recall curves and PR-AUC for all 10 models, No-PCA (left) vs With-PCA (right).

Inference: Precision-Recall curves stay close to the top-right corner for most models in both panels, confirming strong performance on this moderately imbalanced dataset (62.7% / 37.3%), where PR curves are a more sensitive complement to ROC curves. Naive Bayes and Decision Tree show the most visible leftward sag in the With-PCA panel, consistent with their larger negative accuracy deltas under PCA reported above.

14 Results and Discussion

Across all 10 classifiers, moving from the original 30-feature space to a 10-component PCA-reduced space (95.21% variance retained) produced test-accuracy changes ranging from +0.88 points (SVM, Stacking) to −2.64 points (Naive Bayes, Decision Tree/Random Forest), with Logistic Regression and XGBoost essentially unchanged. The best overall No-PCA model was **Logistic Regression** (98.25% accuracy, ROC-AUC 0.9977) and the best overall With-PCA model was **SVM** (98.25% accuracy, ROC-AUC 0.9967) — both linear/margin models, suggesting the WDBC classes are close to linearly separable in both the original and the PCA-rotated space. Ensemble tree methods (Random Forest, Gradient Boosting, AdaBoost, XGBoost) remained strong in both settings but did not exceed the best linear model’s test accuracy here. Training time dropped substantially for tree ensembles under PCA (e.g., Random Forest: 24.1s → 23.0s, Gradient Boosting: 27.7s → 12.6s, AdaBoost: 13.7s → 8.4s), showing a computational benefit of dimensionality reduction even where accuracy is slightly reduced.

15 Observation Questions

1. Which models improved most with PCA? Which did not? Why?

SVM and Stacking improved the most (both +0.88 points in test accuracy). Naive Bayes

showed the largest degradation (-2.64 points), with Random Forest and Decision Tree close behind (-2.63 points each). Margin-based models such as SVM benefit from PCA's decorrelation of the input features (SVM kernels are sensitive to redundant, correlated dimensions inflating certain directions of the feature space). Tree-based models (Random Forest, Decision Tree) already perform their own implicit feature selection by splitting on individual raw features, so compressing/rotating the space via PCA removes some of the specific raw signal these models exploit, causing a small accuracy loss. Gaussian Naive Bayes degraded for a different, more fundamental reason: it explicitly assumes the features are *conditionally independent* given the class, and separately that each feature is Gaussian-distributed within each class. On the original 30 raw features this independence assumption is already violated (the correlation heatmap in Figure 1 shows several `*_mean` features are strongly correlated), so in principle PCA's decorrelated components could be expected to make the independence assumption *more* valid, not less. Empirically here the opposite was observed (-2.64 points): a plausible explanation is that the 10 retained principal components, being linear combinations of all 30 original features, no longer have the same well-separated per-class Gaussian shape that some of the original raw measurements (e.g. `radius_mean`, `area_mean`) had, so Naive Bayes' Gaussian-density assumption is affected even as its independence assumption is somewhat improved. This mixed effect on Naive Bayes' two underlying assumptions is a genuinely model-specific result of this experiment rather than a general rule, and is not directly attributable to PCA "helping" one assumption while leaving raw-feature handling untouched.

2. Did PCA reduce variance across folds / make results more stable?

Only 4 of the 10 models (KNN, Logistic Regression, Random Forest, XGBoost) showed a lower standard deviation across the 5 CV folds under With-PCA versus No-PCA. The remaining 6 models were equally or less stable under PCA. This indicates PCA's effect on cross-validation stability is *model-dependent* rather than a general stabilizing transformation for this dataset – it is not correct to assume dimensionality reduction automatically reduces fold-to-fold variance.

3. For high-dimensional data, was PCA beneficial in reducing overfitting?

With only 30 original features, WDBC is not a very high-dimensional dataset relative to its 455 training samples, so the overfitting-reduction benefit of PCA is modest here. The clearest signal is in Decision Tree, whose CV-mean accuracy (93.63% No-PCA vs 93.41% With-PCA) and test accuracy (96.49% vs 93.86%) both show PCA did *not* meaningfully reduce the CV/test gap for this specific model; if anything the test accuracy fell more than the CV estimate suggested. For genuinely high-dimensional data (hundreds/thousands of features), the overfitting-reduction benefit of PCA is expected to be considerably larger than observed on this 30-feature dataset.

4. How did linear models (Logistic Regression, SVM) compare to ensemble models with PCA?

Under PCA, Logistic Regression reached 98.25% test accuracy and SVM reached 98.25%, both exceeding every ensemble model under PCA (the best ensemble under PCA, XGBoost, reached 97.37%). This shows that once the WDBC features are projected onto their top 10 orthogonal variance-maximizing directions, the classes remain close to linearly separable, so linear models are at least as competitive as – and here slightly ahead of – non-linear ensembles.

5. Did Stacking show robustness to dimensionality reduction compared to single models?

Stacking's accuracy *improved slightly* under PCA ($96.49\% \rightarrow 97.37\%$, $+0.88$ points),

in contrast to two of its own base learners: Naive Bayes fell sharply (-2.64 points) and Decision Tree also fell (-2.63 points) under PCA individually. This suggests the Logistic-Regression meta-learner was able to partly compensate for the PCA-induced degradation in the weaker base learners by leaning more on the SVM base learner (which itself improved under PCA), giving the stacked ensemble greater robustness to the dimensionality-reduction setting than at least two of its individual base models.

16 Conclusion

All 10 classifiers were successfully trained, hyperparameter-tuned with 5-fold stratified cross-validation, and evaluated on a held-out test set, both without PCA (30 original features) and with PCA (10 components, 95.21% variance retained), with scaling and PCA fit only within training folds to avoid leakage. The experiment shows that PCA-based dimensionality reduction on the WDBC dataset preserves most classifiers' discriminative performance reasonably well (median accuracy change of approximately -0.88 percentage points across the 10 models, ranging from $+0.88$ points for SVM/Stacking to -2.64 points for Naive Bayes), with the effect direction and magnitude clearly model-dependent: linear/margin models (SVM) and the heterogeneous Stacking ensemble benefited slightly, while Naive Bayes and single Decision Trees lost the most accuracy. The best-performing configuration overall was Logistic Regression / SVM (both 98.25% test accuracy), underscoring that this dataset is close to linearly separable in both the original and PCA-reduced feature spaces.

17 References

- [1] W.N. Street, W.H. Wolberg and O.L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," *IS&T/SPIE 1993 International Symposium on Electronic Imaging*, 1993.
- [2] Wisconsin Diagnostic Breast Cancer (WDBC) Dataset, UCI Machine Learning Repository.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proc. 22nd ACM SIGKDD*, 2016.
- [5] I.T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer, 2002.